AI Mock Interviewer — Project Spec

Oct 6, 2026 · @Hibah

Overview

A mock interviewer for any job: the user uploads a résumé and job description, picks a length (10, 30, 45 or 60 minutes, or custom), answers out loud, and gets a score plus concrete advice on what to say instead. In the first release the interviewer replies in text; later versions add a spoken interviewer, then (optionally) real-time voice.

- **Who it's for:** anyone job-hunting, in any field — software, sales, marketing, media, business, healthcare, retail, and more.
- **What makes it different:** role-specific interview formats (including role-plays), time-budgeted sessions, rewritten "stronger answers" in the user's own voice, and a story bank that builds up across sessions.
- **Cost rule:** free tiers and open-source software wherever possible. The only paid items are the developer's Claude Pro subscription and Fire API usage for transcription, chat and speech (roughly 1 to 2 cents per minute of full voice; confirm who covers it).
- **Framing:** scores are practice feedback, not a hiring assessment or certification.

Tech stack

Almost every layer is free or open source; the one usage-based service is the Fire API, which handles transcription, the interviewer's replies and its voice. LiveKit is not needed until the optional real-time stage.

Layer	Choice	Cost
Frontend	React + Vite + Tailwind; the browser's MediaRecorder API records each answer	Free (Vercel or Netlify)
In-browser tools	Monaco (code editor), Excalidraw (whiteboard), MediaPipe (body language), all running in the user's browser	Free

Layer	Choice	Cost
Backend API	Flask (Python), the only part that talks to Fire	Free
Speech-to-text	Fire `/v1/transcribe` (Whisper Large v3 Turbo)	Well under 1 cent per minute
Interviewer brain	Fire `/v1/chat`	Part of the 1 to 2 cents per minute total
Interviewer voice (turn-based stage)	Fire `/v1/speech`	Part of the 1 to 2 cents per minute total
Real-time voice (optional later stage)	LiveKit Agents with the LiveKit Cloud free tier	Free tier
Emotion-aware voice (optional experiment)	Hume EVI in your own Hume account	Free tier is 5 minutes; Starter is $3/month for 40 minutes
Database	MySQL	Free
PDF parsing and export	pypdf / pdfplumber to read; WeasyPrint or browser print to export	Free
Hosting	Oracle Cloud Always Free virtual machine (backend and database); Vercel or Netlify (frontend)	Free
Domain and HTTPS	Free subdomain (DuckDNS) with Caddy for automatic HTTPS	Free
Coding help	Claude Code with Claude Pro	Already paid

Free-tier notes:

- Oracle's free ARM allowance is now 2 cores and 12 GB of RAM, plenty now that speech runs on Fire instead of the server. Set a $1 budget alert.
- Avoid per-minute voice platforms (Vapi, Retell), phone numbers (Twilio), and paid voices (ElevenLabs).

Voice roadmap and Fire API

Voice arrives in stages, each adding one piece, so the interview logic can be perfected in text before voice adds cost and delay. Stages 1 to 3 use only Fire.

Stage	User	Interviewer	Calls per turn	Why
1. Text (development only)	Types	Text	Fire chat	Build and debug the interview brain
2. Speak in, text out (first release)	Speaks	Text	Fire transcribe + chat	Cheapest and fastest; users practice speaking
3. Turn-based voice	Speaks	Speaks	Fire transcribe + chat + speech	About 1 to 2 cents per minute; 1 to 3 second pause per turn; no interruptions
4. Real-time voice (optional)	Speaks	Speaks and can interrupt	LiveKit agent	True interruptions, peer practice

Hume EVI is a separate experiment: try its web playground on the free minutes, and add it as an optional emotion-aware interviewer only if the emotion modeling clearly improves feedback.

Fire endpoints (full list at https://fire.prosaga.net/v1/capabilities):

- `POST /v1/transcribe` takes JSON `{"audio": "data:audio/webm; base64,..."}` or an https URL, up to about 25 MB per clip, and returns `{"text": ...}`.
- `POST /v1/chat` produces the interviewer's questions, follow-ups, scoring and rewrites.
- `POST /v1/speech` produces the interviewer's spoken voice from stage 3.

Key handling:

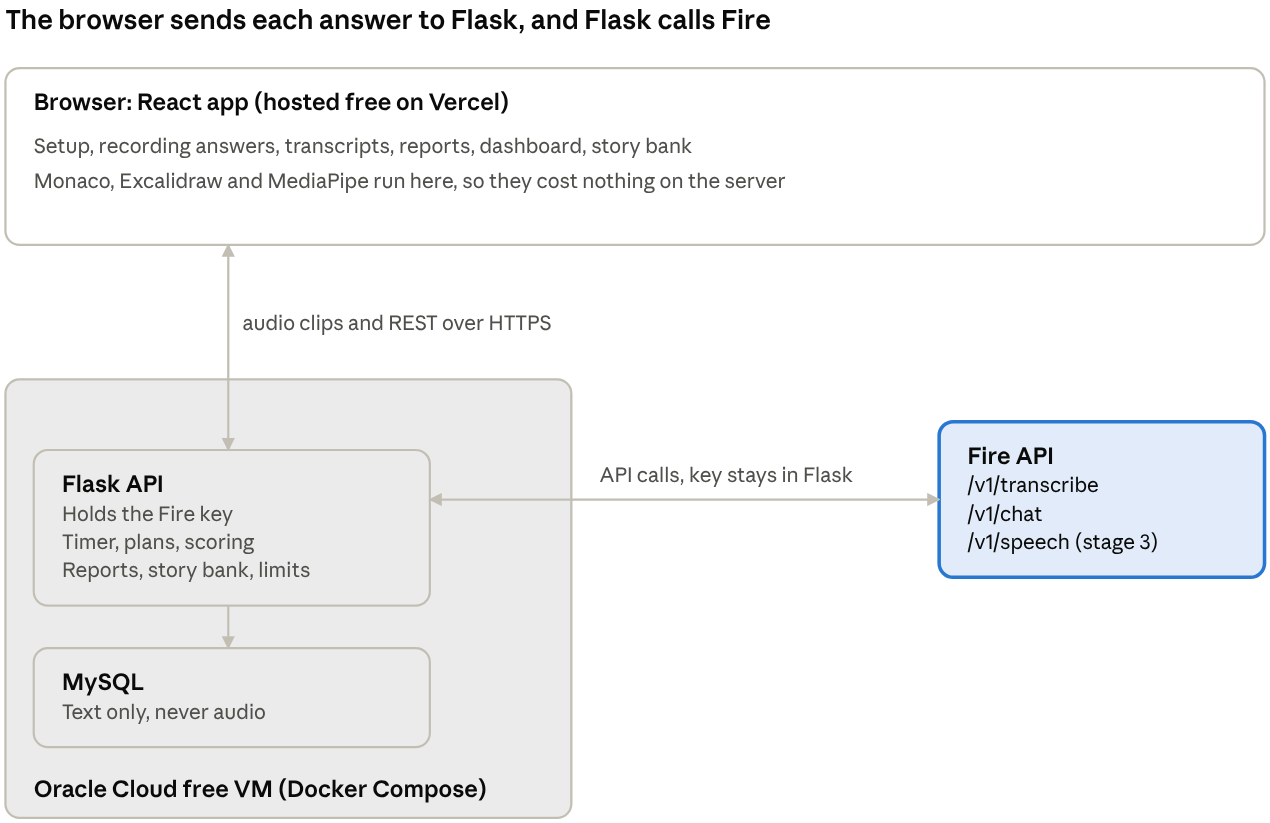
- Use the token issued with chat, speech and transcribe access. The self-made portal token only has chat and image, so it can't call transcribe or speech.
- The token lives only in the backend `.env` as `FIRE_API_KEY`. Add `.env` to `.gitignore` before the first commit.
- The browser never calls Fire. It sends audio to Flask, and Flask calls Fire.

Known limits of turn-based voice:

- Transcribe returns text only, with no timestamps: answer length and speaking pace come from the clip duration measured in the browser, but pauses inside an answer can't be measured.
- Whisper may drop "um" and "uh," so filler-word counts could read low. Test this in stage 2.
- Neither side can interrupt mid-answer; pressure-mode pushback and timer nudges happen between answers.
- Hide the 1 to 3 second pause with a thinking indicator, and in stage 3 a short pre-recorded "Mm-hm, okay."
- Cap each answer at 5 minutes, which keeps clips far below the 25 MB limit.

Architecture

The browser records each answer and sends it to Flask, which calls Fire for transcription, chat and speech; LiveKit joins only if the optional real-time stage is built.



first-release architecture · 4 parts; LiveKit only in the optional real-time stage

How a session flows:

1. The user uploads a résumé and job description; Flask parses them and asks Fire chat for a role analysis, question plan and rubric.
2. Flask saves the plan in MySQL and starts the session timer.
3. The interviewer's question appears as text (stage 2) or plays as audio from Fire speech (stage 3).
4. The user taps record and answers, then taps done or stops talking for a few seconds; the browser sends the clip and its duration to Flask.
5. Flask sends the clip to Fire transcribe, saves the text and duration, discards the audio, and asks Fire chat for the next turn with the timer's state (section nudges, wrap-up).
6. When the session ends, Flask scores each answer in the background, writes recommendations, updates the story bank, and builds the report.
7. If the real-time stage is built, a LiveKit voice agent replaces steps 3 to 5 with streaming audio and interruptions.

Core features

The core loop is setup → interview → report → retry, with progress tracked across sessions.

Setup

- Upload a résumé (PDF or text) and a job description (paste or PDF).
- Choose a duration: 10, 30, 45 or 60 minutes, or a custom slider from 5 to 60.
- Choose a mode: behavioral, technical, case, role-play, system design, coding, or a full mixed loop. Only modes that fit the job are offered.
- Choose difficulty (entry, mid, senior) and interviewer persona (friendly, neutral, tough).
- Preview the generated question plan, with minutes per section, and edit it before starting.

During the interview

- The interviewer replies in text in the first release, then as spoken audio in the turn-based voice stage.

- The user answers by voice with a record button: tap when done, or auto-stop after a few seconds of silence. Typing is available as a fallback.
- A thinking indicator, and later a short spoken "Mm-hm," covers the 1 to 3 second processing pause.
- Adaptive follow-ups that probe vague answers.
- Backend timer with section nudges ("4 minutes left in this section") and a graceful wrap-up, delivered between answers.
- Bonus questions from a reserve list if the candidate finishes early; polite redirects ("Let's keep the next one shorter") if they ramble.
- "Repeat the question," "Can I have a hint?" (hints are noted in the score), pause and resume.
- Visible countdown and current section, with an option to hide it.
- The transcript of each answer appears after it's processed, so the user can check what was heard.
- Coding round (Monaco) and whiteboard (Excalidraw), shown only for roles that need them; their contents are sent along with the answer.
- Interrupting the interviewer, and the interviewer cutting in, arrive only with the optional real-time stage.

Report

- Score per question against the role's rubric, plus an overall "hire / lean hire / no hire" verdict with reasoning.
- Transcript with weak moments highlighted.
- Speech analytics: time per answer versus its budget, and speaking pace in words per minute from each clip's length. Filler-word counts depend on whether Whisper keeps "um" and "uh"; pauses within an answer need the real-time stage.
- STAR-structure analysis for behavioral answers.
- Report builds question by question in the background, so it fills in progressively.
- PDF export and a shareable link.

Recommendations (what you could have said)

- "What you said" next to "A stronger version," rewritten in the user's voice using only facts from their résumé or answers.

- Never invents achievements: missing details become placeholders such as "[add the % increase you achieved]".
- Highlighted changes with reasons: added, cut, reframed.
- Checklist of points a strong answer would have covered, based on the rubric.
- Delivery tips (fillers, length, trailing off) and tips specific to the field.
- Top 3 takeaways at the top of the report.
- "Try again" on any question, with the before-and-after score shown.

Progress

- Dashboard of scores over time, broken down by competency.
- Weak-spot sessions generated from past reports.
- Personal question bank with spaced repetition: missed questions come back later.
- Practice streaks and goals.

Any-job support

The app supports any job by mixing a small set of reusable interview format modules, chosen per job from the job description.

Step 1: role detection. The LLM reads the job description and returns the field, seniority, key competencies, and the formats that fit.

Step 2: format modules. Each module is a reusable prompt and agent behavior:

Module	What happens	Typical fields
Behavioral (STAR)	"Tell me about a time…" with follow-ups	Every job
Résumé deep-dive	Probes specific résumé lines and gaps against the job description	Every job
Technical Q&A	Knowledge questions for the field	Software, data, finance, engineering
Coding	Live problem in Monaco; agent discusses the code	Software, data

Module	What happens	Typical fields
System design	Whiteboard in Excalidraw; agent reviews the design	Software
Case	Market sizing, launch plans, metrics problems	Business, consulting, marketing, product
Role-play	Agent becomes a character (skeptical buyer, angry customer, recruiter)	Sales, customer service, retail, hospitality
Scenario	"What would you do if…" judgment questions	Healthcare, teaching, management, trades
Portfolio walkthrough	User explains past work; agent probes decisions	Marketing, media, design

Adding a new field means choosing a new mix of modules, not writing new code.

Step 3: rubrics per role. Each session gets a rubric built from the job description's competencies (sales: rapport, objection handling, closing; nursing: patient safety, communication, judgment), plus universal criteria (structure, clarity, relevance). The rubric is shown in the report so scores feel fair.

Role-play rule: the agent clearly announces when it switches into a character and when it switches back to debrief.

Accuracy caveat: LLM scoring is weaker in specialized or licensed fields (healthcare, law, accounting). Those reports carry a short disclaimer that feedback is for practice only.

Rollout order: software first (easiest to judge quality), then sales and customer service role-plays, then all other fields.

Signature features

These six features set the project apart; the story bank is the headline.

1. Story bank

- After every session, the LLM extracts reusable stories from the user's answers ("fixed the payment bug," "calmed an angry client") with a title, a STAR summary, and the competencies each one demonstrates.

- Duplicate stories across sessions are merged, keeping the strongest version.
- A coverage view maps stories to competencies and flags gaps: "No story for conflict or failure."
- Before an interview, the app suggests which stories fit the job description.
- Users can edit, rename, or delete any story.
- Free: extraction is one LLM call per session, run in the background.

2. Post-interview debrief (no recordings, ever)

- After a real interview, the user recalls the questions and their answers from memory, by typing or by speaking.
- **Strictly no recordings:** the app never accepts audio or video uploads of a real interview. Spoken recall is transcribed live and the audio is discarded immediately; only the user's text is saved.
- The app rebuilds the interview, scores each answer, writes stronger versions, and predicts likely next-round questions.
- One-click "Practice the next round" builds a session from the debrief.
- A visible notice explains why: recording someone without consent is illegal in some places.

3. Reverse mode (you're the interviewer)

- The AI plays a candidate for the user's chosen job: strong, average, weak, or exaggerating their résumé.
- The user asks questions, then makes a hire / no-hire decision with reasons.
- The app reveals the candidate's hidden profile and scores the user's questions and judgment.
- Useful for job seekers (hearing weak answers from the outside) and new managers practicing hiring.

4. Pressure mode

- Difficulty levels that add a skeptical interviewer, time limits ("30 seconds"), curveball questions, and pushback ("That's not what I asked").
- In the turn-based stages, pressure comes between answers; mid-answer interruptions need the real-time stage.

- Tough but always respectful: no insults or personal attacks, enforced in the system prompt.
- Tracks a composure score: how much answer quality holds up under pressure compared with the user's normal sessions.
- User can stop or lower the pressure at any time.

5. Peer practice

- Needs two people talking live, so it's the one feature that requires the real-time stage with LiveKit.
- Two people join one LiveKit room by invite link; one interviews, one is the candidate, then they can swap.
- The AI listens silently, scores the candidate, and coaches the interviewer on question quality.
- Both people must accept a consent screen before audio is transcribed; nothing is recorded or stored as audio.
- The person interviewing gets suggested questions from the candidate's job description.
- Cheap to run: the AI only listens and summarizes, so it uses fewer tokens than a normal session.

6. Job description decoder

- Reads between the lines of a job description: likely team size, pace, real priorities, red flags, and green flags.
- Example: "'Stakeholders' appears 6 times, so expect communication questions."
- Lists the questions likely to be asked and the competencies to prepare.
- Clearly labeled as interpretation, not fact.
- Works without signing in, as a shareable entry point to the app.

Extra features

Seven smaller features, all free because they run in the browser or need only a single LLM call.

Feature	What it does	Why it stays free
Body language feedback	Optional camera check of eye contact, posture, and fidgeting	MediaPipe runs in the browser; video never leaves the device
Panel interview	Two or three interviewers (hiring manager, peer, HR) with different voices	Kokoro includes multiple local voices
Résumé gap analysis	Shows which job requirements the résumé covers and which questions the gaps will trigger	One LLM call
Company research mode	User pastes the company's website link; the app reads the public About or Careers page and asks "Why us?" questions	Simple page fetch plus one LLM call
Negotiation role-play	Agent plays a recruiter making a low offer; scored on anchoring, justification, and staying calm	Same voice pipeline
Questions-for-them coach	Helps prepare smart questions to ask at the end and rates the ones asked	One LLM call
Cheat sheet and thank-you email	One-page PDF of best stories, key numbers, and weak spots; a drafted thank-you email from the session	One LLM call plus PDF export

Database schema

Thirteen MySQL tables cover every feature; flexible LLM output (plans, rubrics, analytics) is stored as JSON columns. No table stores audio or video.

Table	Key columns	Notes
users	id, email, password_hash, name, created_at	Auth via Flask + JWT

Table	Key columns	Notes
documents	id, user_id, kind (resume / job_description), raw_text, parsed_json, expires_at	Original files deleted after parsing
jd_analyses	id, document_id, field, seniority, competencies_json, formats_json, decoder_json	Shared by setup and the decoder
sessions	id, user_id, resume_id, jd_id, mode (standard / reverse / pressure / peer / debrief), duration_min, difficulty, persona, pressure_level, plan_json, rubric_json, status, started_at, ended_at	One row per interview
questions	id, session_id, section, order_no, text, budget_sec, is_bonus	The question plan
answers	id, question_id, transcript, started_ms, ended_ms, attempt_no, hints_used	attempt_no supports "Try again"
evaluations	id, answer_id, score, rubric_scores_json, missing_points_json, stronger_version, changes_json, delivery_json	Recommendations live here
reports	id, session_id, overall_score, verdict, top_takeaways_json, analytics_json, share_token	share_token for public links
stories	id, user_id, title, star_json, competencies_json, source_answer_ids_json, strength	Story bank
debriefs	id, user_id, company, role, round, recalled_qa_json, analysis_json	Text only, no recordings
peer_rooms	id, host_user_id, guest_user_id, invite_code, consent_host, consent_guest, session_id	Peer practice
question_bank	id, user_id, question_text, competency, last_score, next_review_at	Spaced repetition
usage_limits	user_id, date, minutes_used, llm_calls, sessions_count	Free-tier guard

Privacy, safety, and free-tier guards

The app stores text only, asks for consent before anyone else is heard, and caps usage so free tiers never run out.

Privacy

- No audio or video is ever stored. Speech is transcribed live and discarded.
- No uploads of real-interview recordings are accepted (debrief is recall-only).
- Camera analysis runs entirely in the browser and is off by default.
- Résumés and job descriptions auto-delete after 30 days unless the user saves them.
- "Delete my data" removes everything for that user in one click.
- Shared report links can be revoked.

Safety

- Pressure mode and role-plays stay respectful: no insults, slurs, or personal attacks.
- Stronger-answer rewrites never invent experience; placeholders mark missing facts.
- Disclaimers on scores for licensed and specialized professions.
- The JD decoder labels its output as interpretation.

Usage and cost guards

- Daily caps per user, tracked in usage_limits. Suggested starting caps: 60 interview minutes, one 60-minute session, 5 decoder runs.
- Fire spend estimated per user from minutes used, with a monthly ceiling that pauses new sessions if reached.
- Defaults are 10 and 30 minutes, and stage 2 (text replies) stays available as a cheaper option after spoken replies arrive.
- The Fire key stays on the server; every Fire call goes through Flask. Audio clips sent to Fire are discarded after transcription.
- Oracle budget alert at $1; no paid add-ons enabled in any account.
- A small admin page shows daily usage and estimated Fire spend.

Build phases with Claude Code

Build in phases, one feature per Claude Code session, committing to git after each. Each phase ends with something that works on its own, and voice is added one stage at a time. Phases 1 to 10 use only Fire; 11 and 12 are optional.

1. **Foundation:** repo layout, `CLAUDE.md`, `.env` and `.gitignore`, Flask app, MySQL schema, sign-up and login, React shell.
2. **Text-only brain (voice stage 1):** résumé and job description upload → JD analysis → question plan with time budgets → rubric, using Fire chat. Typed answers both ways.
3. **Speak in, text out (voice stage 2, first release):** record button, silence auto-stop, Flask → Fire transcribe → Fire chat, timer and all durations.
4. **Report and recommendations:** scores, stronger versions, missing points, delivery analytics, top 3 takeaways, "Try again."
5. **Any-job support:** role detection, format modules, role-play, per-role rubrics. Software first, then sales and customer service.
6. **Turn-based voice (voice stage 3):** Fire speech for the interviewer, persona voices, latency tricks.
7. **Progress and story bank:** dashboard, story extraction and coverage view, question bank with spaced repetition.
8. **Signature modes:** JD decoder, debrief, reverse mode, pressure mode (between-answer pressure).
9. **Extras:** coding round, whiteboard, panel, body language, gap analysis, company research, negotiation, cheat sheet, thank-you email.
10. **Deploy and harden:** Oracle VM with Docker Compose and Caddy, frontend on Vercel, usage and cost caps, evaluation harness (AI candidates across fields), privacy controls.
11. **Optional real-time voice (voice stage 4):** LiveKit agent, interruptions both ways, peer practice, mid-answer pressure.
12. **Optional Hume experiment:** compare an emotion-aware interviewer against Fire voice; keep it only if it clearly improves feedback.

Working within Claude Pro limits (Sonnet only, roughly 10 to 40 Claude Code prompts every 5 hours):

- Keep `CLAUDE.md` current so each session starts with full context.
- Use plan mode for each phase, approve the plan, then implement.
- Ask for small, testable changes; review and commit before the next prompt.

- Do repetitive work (prompt tuning, copy edits) by hand to save quota for real coding.